

ENSF 380

TOPIC 11: OBJECT ORIENTED PROGRAMMING PART II

Object Oriented Programming (OOP)

Encapsulation

Abstraction

Inheritance

Polymorphism

Direct access to components is restricted

Data and the methods that operate on it are bundled

```
public class Clothing {  
  
    private String colour;  
    private String material;  
  
    // Getters and setters  
    public String getColour() { return this.colour; }  
    public String getMaterial() { return this.material; }  
    public void setColour(String colour) { this.colour = colour; }  
    public void setMaterial(String material) { this.material = material; }  
  
    // Constructor  
    public Clothing(String colour, String material) {  
        this.colour = colour;  
        this.material = material;  
    }  
  
    public static void main(String[] args) {  
        var garment = new Clothing("black", "cotton");  
        System.out.println("Colour: " + garment.getColour() +  
            ", Material: " + garment.getMaterial());  
    }  
}
```

Breaking Encapsulation

02_NotEncapsulated

Direct access to object components is **not** restricted

```
public class Clothing {  
  
    public String colour;  
    public String material;  
  
    // Getters and setters  
    public String getColour() { return this.colour; }  
    public String getMaterial() { return this.material; }  
    public void setColour(String colour) { this.colour = colour; }  
    public void setMaterial(String material) { this.material = material; }  
  
    // Constructor  
    public Clothing(String colour, String material) {  
        this.colour = colour;  
        this.material = material;  
    }  
    public static void main(String[] args) {  
        var garment = new Clothing("black", "cotton");  
        System.out.println("Colour: " + garment.colour +  
            ", Material: " + garment.material);  
    }  
}
```

Encapsulation

Task: Match colour to database colour ID, output colour name as text

```
public class Clothing {
    private int colour;
    private String material;
    private final static String COLOURS[] = {"black", "white", "yellow"};

    public String getColour() { return COLOURS[this.colour]; }
    public String getMaterial() { return this.material; }
    public void setMaterial(String material) { this.material = material; }
    public void setColour(String colour) {
        for (int i=0; i<COLOURS.length; i++) {
            if (colour.equals(COLOURS[i])) {
                this.colour = i;
            } else {
                this.colour = 0;
            }
        }
    }

    public Clothing(String colour, String material) {
        this.setColour(colour);
        this.material = material;
    }

    public static void main(String[] args) {
        var garment = new Clothing("black", "cotton");
        System.out.println("Colour: " + garment.getColour() +
            ", Material: " + garment.getMaterial());
    }
}
```

Tip: Avoid duplication by putting logic associated with data checking in one location.

Encapsulation

04_Extension

```
...  
// Getters and setters  
public String getColour() { return COLOURS[this.colour]; }  
public int getColourID() { return this.colour; }  
public String getMaterial() { return this.material; }  
public void setMaterial(String material) { this.material = material; }  
public void setColour(int colour) {  
    if (colour < COLOURS.length) {  
        this.colour = colour;  
    } else {  
        this.colour = 0;  
    }  
}  
  
// Constructors  
public Clothing(String colour, String material) {  
    this.setColour(colour);  
    this.material = material;  
}  
public Clothing(int colour, String material) {  
    this.setColour(colour);  
    this.material = material;  
}  
...
```

Important: You cannot have two methods with the same name which differ only by return type.

Encapsulation

Direct access to components is restricted

Data and the methods that operate on it are bundled

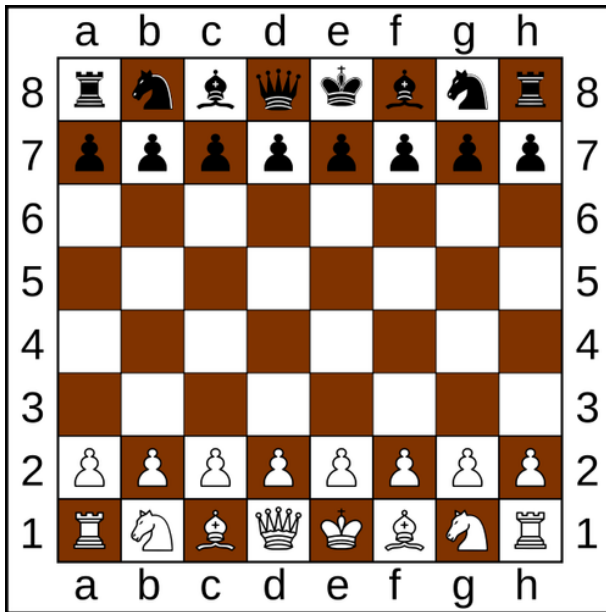
```
public class Pants {
    private Clothing fabric;
    private int waistSize;
    private int legSize;

    public Pants(String colour, String material, int waist, int leg) {
        this.fabric = new Clothing(colour, material);
        this.waistSize = waist;
        this.legSize = leg;
    }

    public static void main(String[] args) {
        var myPants = new Pants(1, "cotton", 34, 36);
        System.out.println("Colour: " + myPants.getColour() + ", Material: " +
            myPants.getMaterial() + ", Waist: " + myPants.getWaistSize() +
            ", Leg: " + myPants.getLegSize());
    }
}
```

Encapsulation

chessBoard



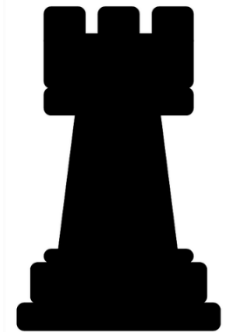
Knight

2 on x, 1 on y (jumps)
1 on x, 2 on y (jumps)



Queen

any distance x
any distance y
any distance diagonally



Rook

any distance x
any distance y
castle (jumps)

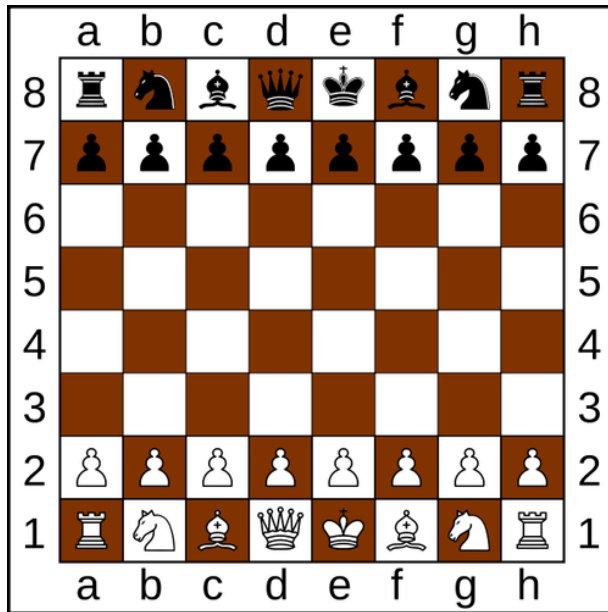
`bool canMove(int origNumCord, char origLetterCord, int newNumCord, char newLetterCord)`

Encapsulation

`bool canMove(int numCord, char letterCord, bool isJump)`

chessBoard

piece at location
new location is on board
piece at new location
move requires jump



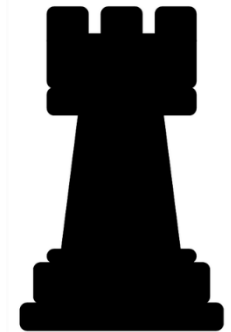
Knight

2 on x, 1 on y (jumps)
1 on x, 2 on y (jumps)



Queen

any distance x
any distance y
any distance diagonally



Rook

any distance x
any distance y
castle

Exercise 11.1

TEST YOUR UNDERSTANDING BY COMPLETING THE EXERCISE

Abstraction

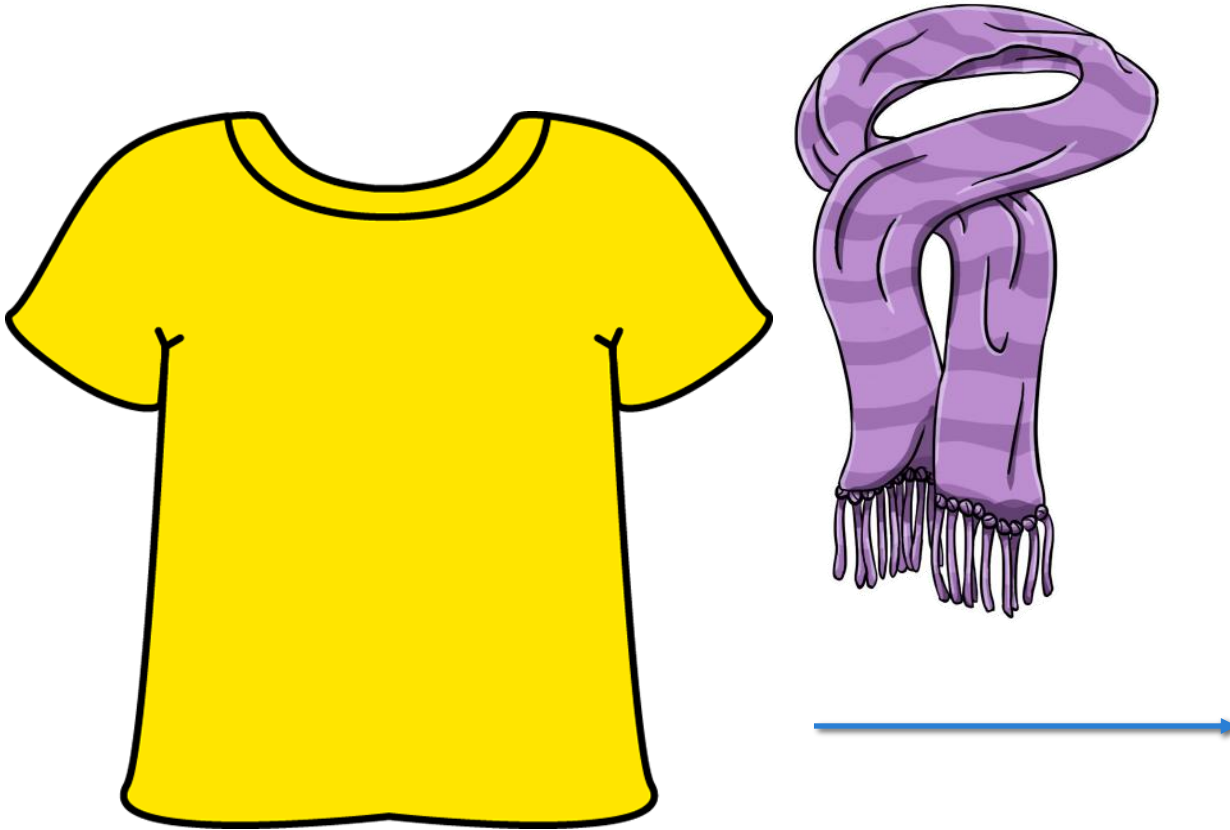
All but essential details are removed

Complexity is reduced

Abstraction



TShirt	
-	price
-	fit
-	size
-	colour
-	description
-	condition
+	setPrice(bulkPrice double)
+	getPrice() double
...	



Garment
- price - brand - size - colour - description - condition
+ setPrice(bulkPrice double) + getPrice() double ...

Exercise 11.2

TEST YOUR UNDERSTANDING BY COMPLETING THE EXERCISE

Exercise

Exercise 11.2

1. Determine which attributes you would include for an abstraction of "plant" from the perspective of a garden center.

Potential attributes:

blossomColour

bulkPrice

clade

commonName

cultivar

dateReceived

family

floweringMonth

genus

kingdom

needleLength

order

originatingNursery

pollenCount

preferredAcidity

preferredLight

retailPrice

species

toxicity

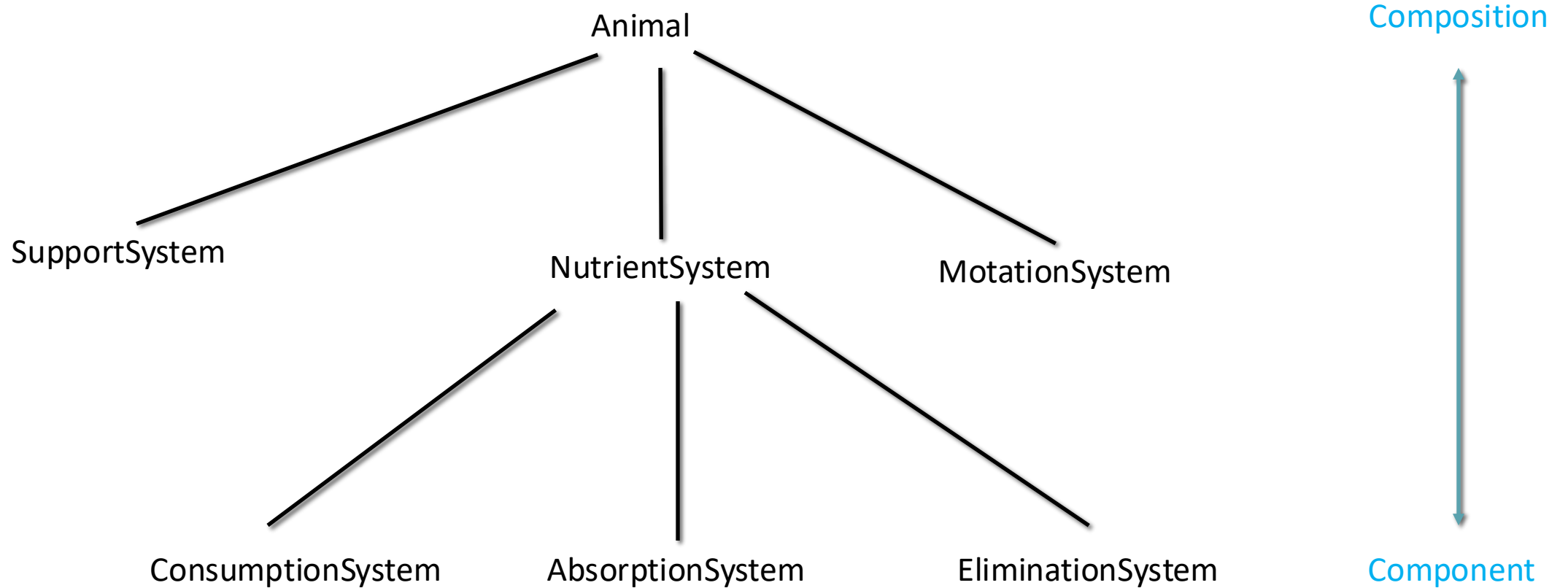
wateringFrequency

Inheritance

Method of deriving one class from another

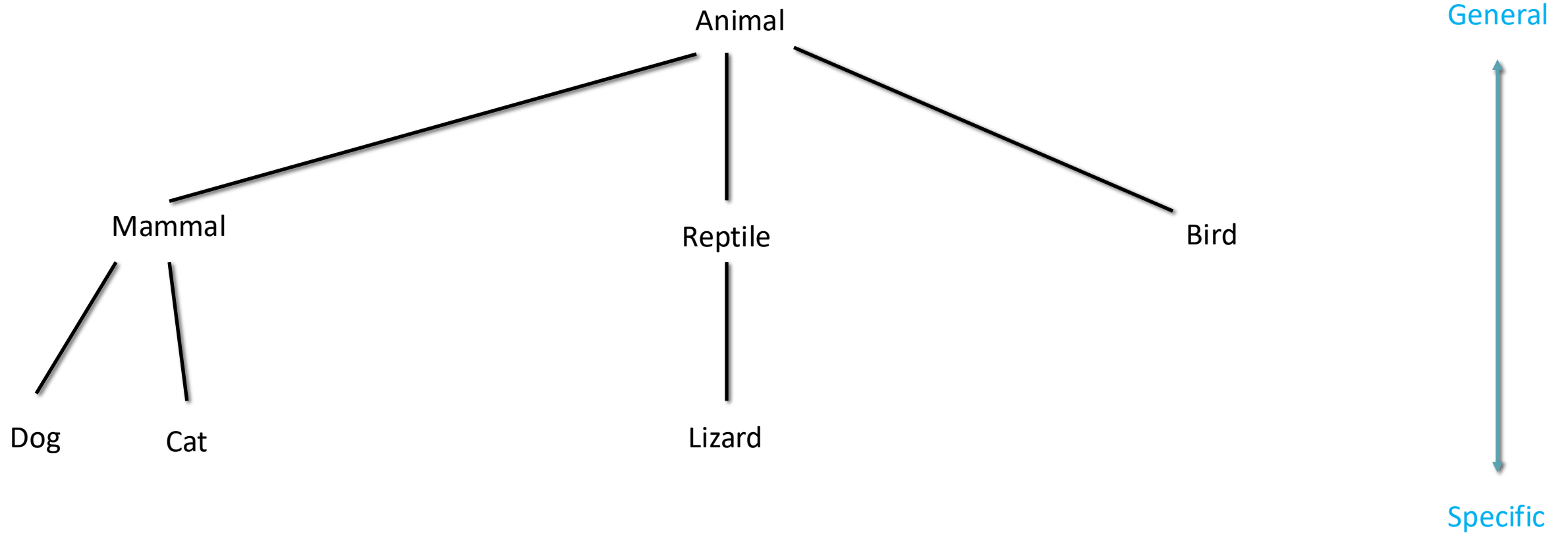
Hierarchy

"has a"



Hierarchy

"is a"



Exercise 11.3

TEST YOUR UNDERSTANDING BY COMPLETING THE EXERCISE

Object-Oriented?

06_NotOO

Object-Oriented Calculator

```
public class CalculatorOO {
    private double one;
    private double two;

    public double add() { return one + two; }
    public double subtract() { return one - two; }
    public double multiply() { return one * two; }
    public double divide() {
        if (two == 0) { return 0; }
        return one/two;
    }

    // Constructor
    public CalculatorOO(double one, double two) {
        this.one = one;
        this.two = two;
    }
}
```

Calculator (Not Object-Oriented)

```
public class CalculatorNotOO {

    public static double add(double one, double two) {
        return one + two;
    }
    public static double subtract(double one, double
+ two) {
        return one - two;
    }
+   public static double multiply(double one, double
two) {
        return one * two;
    }
    public static double divide(double one, double two)
    {
        if (two == 0) { return 0; }
        return one/two;
    }
}
```

Benefits of OOP

Modularity

Reuse

Flexibility

Exercise 11.4

TEST YOUR UNDERSTANDING BY COMPLETING THE EXERCISE

Exercise 11.5

TEST YOUR UNDERSTANDING BY COMPLETING THE EXERCISE

Polymorphism

Static Polymorphism

Dynamic Polymorphism

- Different objects accessed through the same interface, but different implementation
- Works when classes are related in the inheritance tree (common superclass)
- Subclass redefines the method to specialize the behavior

Summary: Knowledge

The main concepts of OOP

- Encapsulation and how and why to use it
- Abstraction and how and why to use it
- Inheritance
- Polymorphism

Two types of hierarchies

- "has a"
- "is a"

Benefits of OOP

Summary: Skills

Applying different encapsulation paradigms and analyzing the impact

Determining appropriate abstraction requirements

Identifying different types of hierarchies

Recognizing OOP

Additional Information

Related reading

- Volume 1, Chapter 4.1.1: Classes
- Volume 1, Chapter 4.1.1: Objects
- Volume 1, Chapter 4.3.8: Benefits of Encapsulation